

HOSTEL ACCOMMODATION AND STUDENT SERVICES MANAGEMENT SYSTEM

Presented By

Kartikeya Parcha

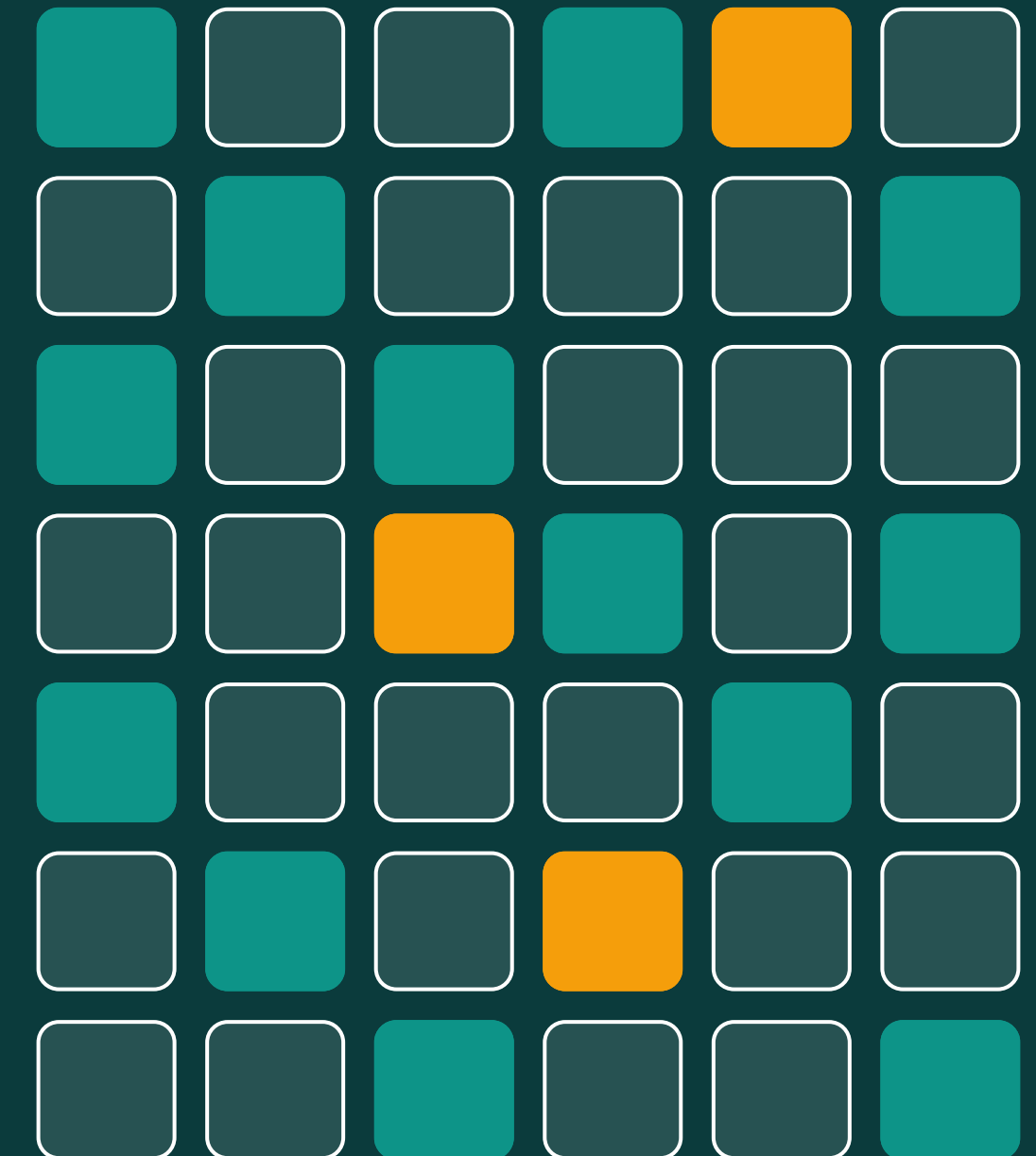
Roll No: 25WU0102116

B.Tech Artificial Intelligence & Machine Learning

AIML Panthers

Faculty: **Dr. Kiran Mayee Adavala**

Woxsen University · Academic Year 2026–2027



 Occupied

 Under maintenance

Abstract

- A centralized database system for managing university hostel operations.
- Maintains student, hostel, room, bed, allocation and payment information.
- Provides complaint, maintenance and student service management.
- Supports role-based access for Students, Wardens and Administrators.
- Uses MySQL for database management and Python Flask for the web application.
- Reduces manual work and improves data accuracy, accessibility and request tracking.

15

MySQL tables linked by foreign keys

3

role-based portals, each with its own UI

1

central database, updated live from the web app

Introduction & Problem Statement

Introduction

- Hostel management involves large amounts of student and accommodation data.
- Manual record keeping makes information difficult to maintain and retrieve.
- A centralized DBMS can organize hostel operations efficiently.
- The system connects accommodation and student service activities in one platform.

Problem Statement

1

Manual management of student and hostel records

2

Difficulty tracking available rooms and beds

3

Inefficient complaint and maintenance tracking

4

Difficulty maintaining payment records

5

Lack of centralized student service management

6

Delayed status updates and information retrieval

Objectives & Scope

Objectives

- 1 Maintain student information digitally
- 2 Manage hostels, rooms and beds
- 3 Allocate beds to students
- 4 Maintain payment records
- 5 Manage student complaints
- 6 Track maintenance requests
- 7 Manage student service requests
- 8 Maintain visitor and vacating records
- 9 Provide role-based access

Scope

S Student

- Payments
- Complaints
- Maintenance
- Service requests
- Accommodation details

W Warden

- Student management
- Bed allocation
- Complaint management
- Maintenance management
- Service management

A Administrator

- Student records
- Payment records
- Complaint/service monitoring

Software & Hardware Requirements

Software Requirements

Component	Technology
Operating System	Windows
Programming Language	Python
Backend	Flask
Database	MySQL
Database Tool	MySQL Workbench
Frontend	HTML, CSS
IDE	VS Code
Browser	Chrome / Edge

Hardware Requirements

- 1

Laptop / Desktop
- 2

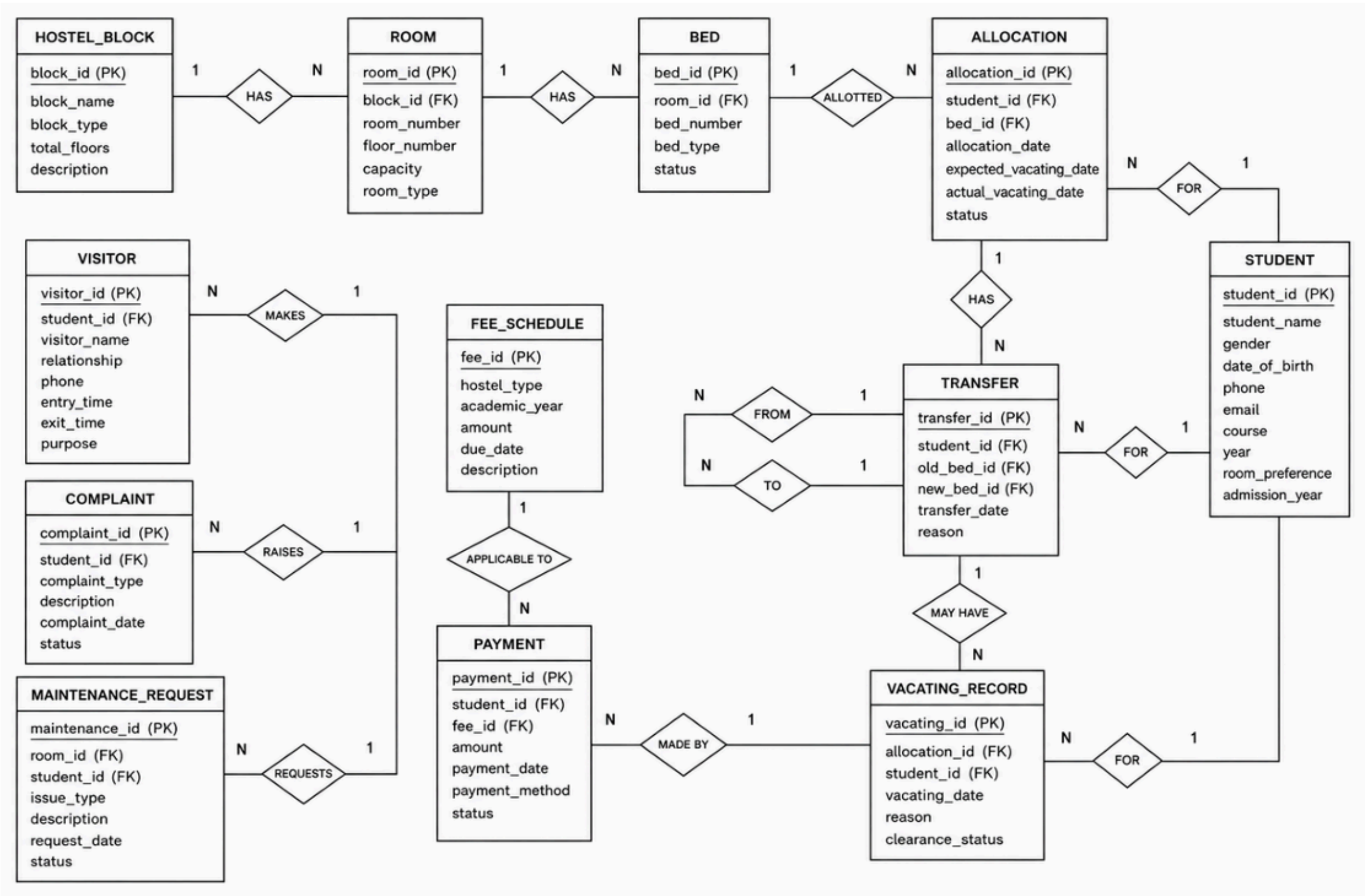
Minimum 8 GB RAM
- 3

Minimum 256 GB storage
- 4

Internet / LAN connectivity
- 5

Standard keyboard and mouse

ER Diagram



Key Relationships

Hostel → Rooms

Room → Beds

Student → Allocation

Student → Payments

Student → Complaints

Student → Maintenance Requests

Student → Service Requests

Student → Visitors

Student → Vacating Records

1 : N relationships; allocations link students to beds.

Relational Schema & Normalization

Students

student_id, student_name, gender, date_of_birth, phone, email, course, year, room_preference, admission_year

Rooms

room_id, block_id, room_number, floor_number, capacity, room_type

Allocations

allocation_id, student_id, bed_id, allocation_date, expected_vacating_date, actual_vacating_date, status

Hostels

block_id, block_name, block_type, total_floors, description

Beds

bed_id, room_id, bed_number, bed_type, status

Payments

payment_id, student_id, fee_id, amount, payment_date, payment_method, status

Additional relations

Complaints

MaintenanceRequests

Services

ServiceRequests

Staff

Visitors

FeeSchedule

Transfers

VacatingRecords

Normalization

1NF

Atomic values, no repeating groups

2NF

All non-key attributes depend on the complete primary key

3NF

No transitive dependency between non-key attributes; reduces redundancy and update anomalies

Data Dictionary

Table	Primary Key	Important Fields
students	student_id	student_name, email, phone, course
hostels	block_id	block_name, block_type, total_floors
rooms	room_id	block_id, room_number, floor_number
beds	bed_id	room_id, bed_number, status
allocations	allocation_id	student_id, bed_id, dates, status
transfers	transfer_id	student_id, old_bed_id, new_bed_id
vacating_records	vacating_id	student_id, vacating_date, clearance
fee_schedule	fee_id	academic_year, amount, due_date

Table	Primary Key	Important Fields
payments	payment_id	student_id, fee_id, amount, status
complaints	complaint_id	student_id, type, description, status
maintenancerequests	maintenance_id	student_id, room_id, issue, status
services	service_id	service_name, description, charge
servicerequests	request_id	student_id, service_id, status
staff	staff_id	name, role, email
visitors	visitor_id	student_id, visitor_name, visit_date

Constraints used

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

DEFAULT

AUTO_INCREMENT

SQL Commands Used

DDL Data Definition

```
CREATE DATABASE hosteldb;

CREATE TABLE students (...);
CREATE TABLE hostels (...);
CREATE TABLE rooms (
    ...
    FOREIGN KEY (block_id)
        REFERENCES hostels(block_id)
);

ALTER TABLE ...;
```

DML Data Manipulation

```
INSERT INTO complaints
    (student_id, complaint_type,
     description)
VALUES (1, 'Water',
        'No hot water');

UPDATE complaints
SET status = 'Resolved'
WHERE complaint_id = 1;

DELETE FROM visitors
WHERE visitor_id = 3;
```

DQL Data Query

```
SELECT * FROM students;

SELECT * FROM rooms;

SELECT * FROM payments;

SELECT * FROM beds
WHERE status = 'Available';
```

```
mysql> SHOW TABLES;    -- 15 tables in hosteldb
```

allocations	maintenancerequests	staff
beds	payments	students
complaints	rooms	transfers
fee_schedule	servicerequests	vacating_records
hostels	services	visitors

Queries with Outputs

Display all students

```
SELECT student_id, student_name,
       gender, course
FROM students LIMIT 3;
```

student_id	student_name	gender	course
1	Aarav Sharma	Male	B.Tech AI/ML
2	Rohan Verma	Male	B.Tech CSE
3	Kiran Reddy	Male	B.Tech AI/ML

Display available beds

```
SELECT bed_id, room_id,
       bed_number, status
FROM beds
WHERE status = 'Available' LIMIT 3;
```

bed_id	room_id	bed_number	status
3	2	B1	Available
4	2	B2	Available
6	3	B2	Available

Display pending complaints

```
SELECT complaint_id, student_id,
       complaint_type, status
FROM complaints
WHERE status IN ('Open','In Progress');
```

complaint_id	student_id	complaint_type	status
1	1	Water	Open
2	4	Noise	In Progress

Display student payments

```
SELECT payment_id, fee_id,
       amount, status
FROM payments
WHERE student_id = 1;
```

payment_id	fee_id	amount	status
1	3	18000.00	Paid
2	1	20000.00	Paid

Display active allocations

```
SELECT allocation_id, student_id,
       bed_id, status
FROM allocations
WHERE status = 'Active' LIMIT 3;
```

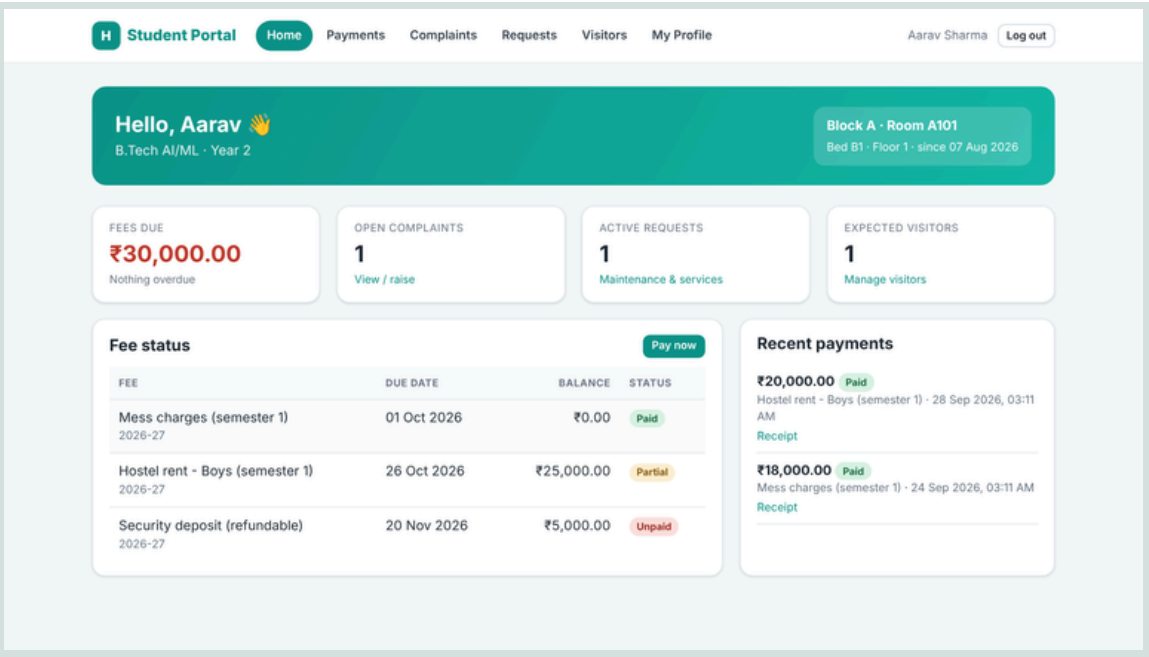
allocation_id	student_id	bed_id	status
1	1	1	Active
2	2	2	Active
3	3	5	Active

Query from Presentation-II

Paste the exact Presentation-II query and its output screenshot here.

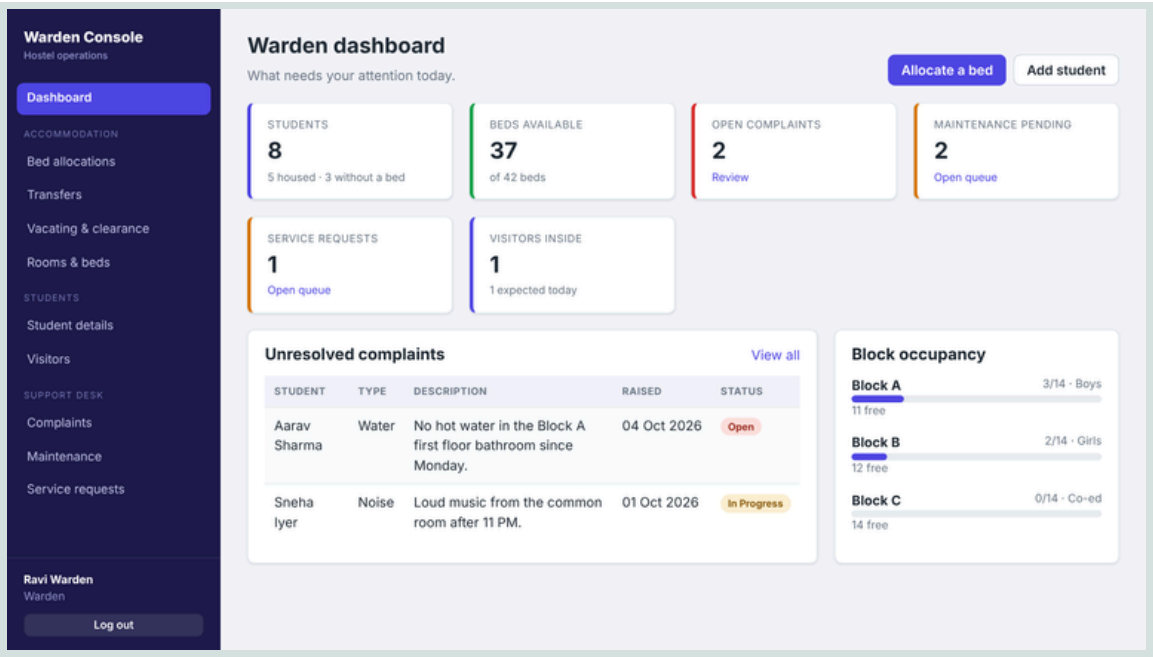
UI Design & Screenshots

Login: username and password authentication · role-based login for Student, Warden and Administrator



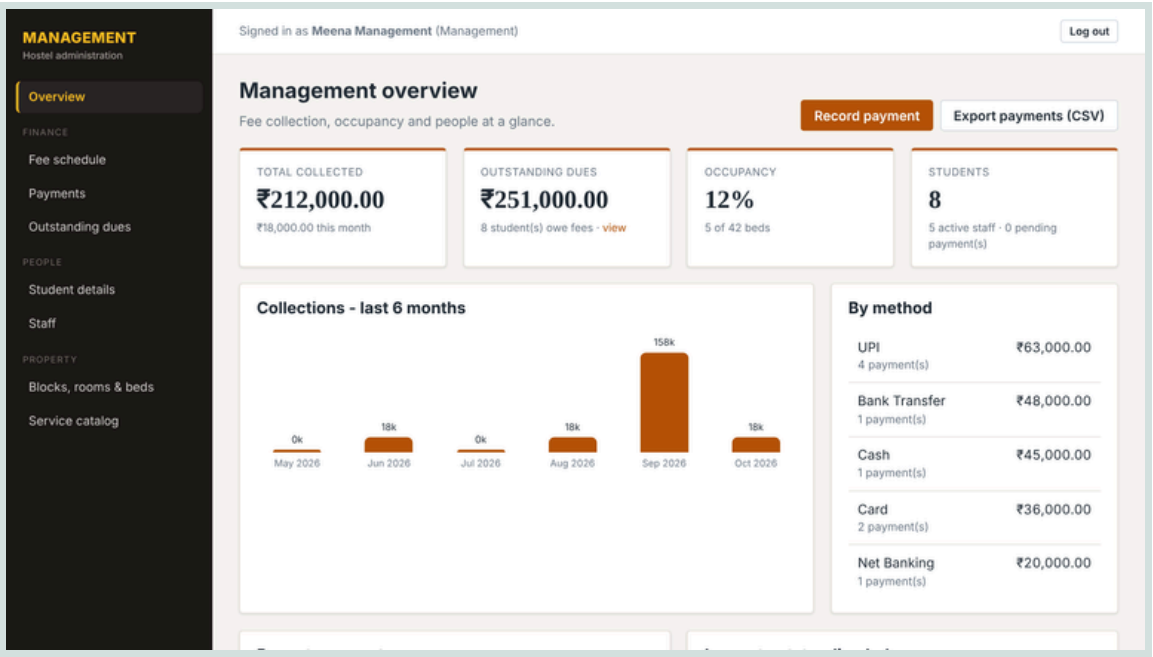
Student Dashboard

- View accommodation
- Make payment
- Submit complaint
- Maintenance request
- Request services



Warden Dashboard

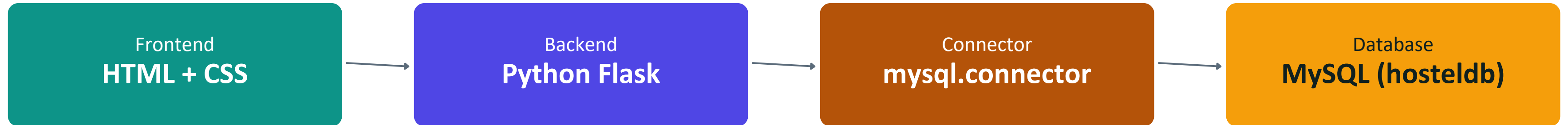
- View and add students
- Allocate beds
- View complaints
- Manage maintenance
- Manage service requests



Administrator Dashboard

- View and edit students
- View payments
- View complaints
- View service requests

Implementation Details



Database connectivity

```
conn = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="*****",  
    database="hostelddb"  
)
```

Key implementation features

- Flask routing
- Session-based authentication
- Role-based access control
- SQL CRUD operations
- Foreign-key relationships
- Dynamic request status updates
- Student → Room/Bed allocation mapping

Request status flow

Complaint

Open

In Progress

Resolved / Rejected

Maintenance

Pending

Assigned

In Progress

Completed / Rejected

Service

Pending

Approved

Completed / Rejected

Testing

Test Case	Expected Result	Result
Student Login	Student dashboard displayed	PASS
Warden Login	Warden dashboard displayed	PASS
Admin Login	Admin dashboard displayed	PASS
Add Student	Student record created	PASS
Bed Allocation	Bed assigned successfully	PASS
Payment	Payment recorded	PASS
Complaint	Complaint created as Open	PASS
Maintenance	Request created successfully	PASS
Service Request	Request created successfully	PASS
Complaint Status	Status updated by Warden	PASS
Maintenance Status	Status updated by Warden	PASS
Service Status	Status updated by Warden	PASS
Invalid Login	Access denied	PASS

13/13

functional test cases passed

23/23

automated checks run against the live MySQL database

24

portal pages rendered without errors

All major functional modules were tested and produced the expected results.

Conclusion & Future Enhancements

Conclusion

- Developed a centralized hostel management system.
- Integrated accommodation and student service management.
- Reduced dependency on manual records.
- Improved data organization and retrieval.
- Implemented role-based access.
- Tracks complaints, maintenance and service requests.
- Maintains relationships using relational database principles.

Future Enhancements

- 1 Online payment gateway
- 2 Email/SMS notifications
- 3 Mobile application
- 4 QR-based visitor management
- 5 Cloud database deployment
- 6 Automated bed availability
- 7 Advanced analytics and reports
- 8 Digital hostel attendance
- 9 AI-based complaint classification

References

Books

- Database System Concepts — Abraham Silberschatz, Henry F. Korth, S. Sudarshan
- Fundamentals of Database Systems — Ramez Elmasri, Shamkant B. Navathe

Documentation

- MySQL Documentation
- Python Documentation
- Flask Documentation
- MySQL Workbench Documentation

Academic Resources

- Woxsen University DBMS Course Material
- Faculty-provided DBMS lecture notes
- SQL and Relational Database Management System references